

## Documentação Técnica – Projeto Meu Controle Financeiro

**Versão:** 1.1

**Data:** 26 de Outubro de 2025

**Autor:** Equipe de Desenvolvimento – Meu Controle Financeiro

**Status:** Estável (Produção)

---

### 1. Visão Geral do Projeto

#### 1.1. Descrição

O “**Meu Controle Financeiro**” é uma aplicação **web full-stack** desenvolvida para auxiliar usuários na **gestão das finanças pessoais**, oferecendo uma visão completa de receitas, despesas, metas e orçamentos mensais.

A plataforma permite:

- Registrar **receitas e despesas** detalhadamente;
- Criar **orçamentos mensais** categorizados;
- Definir **metas financeiras** e acompanhar o progresso;
- Gerar **relatórios e gráficos interativos**;
- Controlar **recorrências** de lançamentos financeiros.

A solução busca oferecer uma interface intuitiva, segura e acessível, com **autenticação JWT** e visualização em **tempo real** via **React**.

---

#### 1.2. Arquitetura da Solução

A aplicação segue o modelo **Cliente-Servidor** desacoplado:

| Camada                    | Descrição                                                                                                                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Frontend (Cliente)</b> | Desenvolvido em <b>React (Vite)</b> . Responsável pela experiência do usuário, comunicação com a API via <b>Axios</b> , roteamento com <b>React Router</b> e estilização com <b>Tailwind CSS</b> . |
| <b>Backend (Servidor)</b> | Desenvolvido em <b>Node.js com Express.js</b> , atua como API <b>RESTful</b> , gerenciando regras de negócio, autenticação, e persistência de dados com <b>Sequelize ORM</b> .                     |
| <b>Base de Dados</b>      | Banco de dados relacional <b>MariaDB</b> , executado em container <b>Docker</b> , garantindo portabilidade e isolamento.                                                                           |

---

#### 1.3. Tecnologias e Ferramentas

| Categoria              | Tecnologias/Ferramentas                                       |
|------------------------|---------------------------------------------------------------|
| Backend                | Node.js, Express.js, Sequelize (ORM), JWT, Bcrypt.js          |
| Frontend               | React (Vite), React Router, Axios, Tailwind CSS, Recharts     |
| Banco de Dados         | MariaDB                                                       |
| Ambiente & Utilitários | Docker, Git & GitHub, VS Code, ESLint, Nodemon, Sequelize-CLI |
| Versionamento          | Git (GitHub)                                                  |
| Testes                 | Jest (planejado para versão 1.2)                              |

---

## 2. Estrutura do Projeto

MEU-CONTROLE-FINANCEIRO/

```

├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── database.js
│   │   ├── controllers/
│   │   │   ├── BudgetController.js
│   │   │   ├── CategoryController.js
│   │   │   ├── GoalController.js
│   │   │   ├── ReportController.js
│   │   │   ├── SessionController.js
│   │   │   ├── SubcategoryController.js
│   │   │   ├── TransactionController.js
│   │   │   └── UserController.js
│   │   ├── database/
│   │   │   ├── migrations/
│   │   │   ├── seeders/
│   │   │   └── index.js
│   │   ├── middlewares/
│   │   │   └── auth.js
│   │   └── models/

```

```
| | └─ routes/
| |   └─ routes.js
| | └─ fonts/
| | └─ app.js
| | └─ server.js
| | └─ .env
| └─ package.json
| └─ docker-compose.yml
|
└─ frontend/
    └─ src/
        └─ components/
        └─ contexts/
        └─ pages/
        └─ services/
        └─ App.jsx
        └─ main.jsx
        └─ index.css
    └─ package.json
    └─ vite.config.js
    |
    └─ Document/      # Documentação do projeto
```

---

### 3. Backend (API – Node.js & Express)

#### 3.1. Estrutura de Pastas

- **config/** – Configurações da aplicação e banco de dados
- **controllers/** – Lógica de negócio e manipulação de dados
- **database/** – Migrations, seeders e inicialização do Sequelize
- **middlewares/** – Interceptadores e validações (ex: autenticação JWT)
- **models/** – Definição de entidades e relacionamentos
- **routes/** – Definição dos endpoints RESTful

---

### 3.2. Modelos de Dados (Sequelize)

| Modelo           | Relacionamentos                               | Descrição                                                      |
|------------------|-----------------------------------------------|----------------------------------------------------------------|
| User             | hasMany(Transaction, Budget, Goal)            | Armazena informações de autenticação e identificação.          |
| Transaction      | belongsTo(User),<br>belongsTo(Subcategory)    | Representa uma receita ou despesa (com suporte a recorrência). |
| Budget           | belongsTo(User),<br>hasMany(Category)         | Define um limite de gasto por categoria.                       |
| Goal             | belongsTo(User),<br>hasMany(GoalContribution) | Representa uma meta de poupança com valor e prazo.             |
| GoalContribution | belongsTo(Goal)                               | Registra contribuições feitas para uma meta.                   |
| Category         | hasMany(Subcategory)                          | Agrupar transações por tipo (Ex: Alimentação, Transporte).     |
| Subcategory      | belongsTo(Category)                           | Subdivisões das categorias principais.                         |

---

### 3.3. Endpoints da API (RESTful)

| Recurso      | Método | Endpoint              | Descrição                       | Protegido |
|--------------|--------|-----------------------|---------------------------------|-----------|
| Autenticação | POST   | /api/register         | Registra novo usuário           | ✗         |
| Autenticação | POST   | /api/login            | Autentica e retorna JWT         | ✗         |
| Usuários     | GET    | /api/user/profile     | Retorna dados do usuário logado | ✓         |
| Transações   | GET    | /api/transactions     | Lista transações do usuário     | ✓         |
|              | POST   | /api/transactions     | Cria nova transação             | ✓         |
|              | PUT    | /api/transactions/:id | Atualiza transação existente    | ✓         |
|              | DELETE | /api/transactions/:id | Exclui transação                | ✓         |
| Orçamentos   | GET    | /api/budgets          | Lista orçamentos mensais        | ✓         |
|              | POST   | /api/budgets          | Cria novo orçamento             | ✓         |
|              | PUT    | /api/budgets/:id      | Atualiza orçamento              | ✓         |

| Recurso    | Método | Endpoint         | Descrição                               | Protegido |
|------------|--------|------------------|-----------------------------------------|-----------|
| Metas      | DELETE | /api/budgets/:id | Remove orçamento                        | ✓         |
|            | GET    | /api/goals       | Lista metas financeiras                 | ✓         |
|            | POST   | /api/goals       | Cria nova meta                          | ✓         |
|            | PUT    | /api/goals/:id   | Atualiza meta existente                 | ✓         |
|            | DELETE | /api/goals/:id   | Remove meta                             | ✓         |
| Relatórios | GET    | /api/reports     | Gera relatórios por categoria e período | ✓         |

### 3.4. Autenticação e Segurança

O sistema utiliza **JWT (JSON Web Token)** com **Bcrypt.js** para criptografia de senhas.

Fluxo de autenticação:

1. Usuário envia **email e senha** → /api/login;
2. Servidor valida credenciais e gera **token JWT**;
3. Token é armazenado no cliente (localStorage);
4. Requisições subsequentes enviam Authorization: Bearer <token>;
5. Middleware auth.js valida o token antes de permitir o acesso.

## 4. Frontend (React + Vite)

### 4.1. Estrutura de Pastas

- **components/** – Componentes reutilizáveis (formulários, gráficos, modais, layout)
- **pages/** – Páginas principais (Dashboard, Metas, Categorias, Relatórios etc.)
- **contexts/** – Contexto global (ex: autenticação com AuthContext)
- **services/** – Comunicação com API usando Axios
- **assets/** – Ícones, imagens e recursos visuais

### 4.2. Funcionalidades Principais

- Login e Registro com persistência de sessão;
- Dashboard interativo com **gráficos Recharts**;
- Cadastro e visualização de **transações, orçamentos e metas**;
- Relatórios por **categoria/subcategoria**;

- Design responsivo com **Tailwind CSS** e **MobileHeader.jsx**.
- 

### 4.3. Comunicação com Backend

A comunicação entre o React e a API é feita via **Axios**.

Exemplo de requisição autenticada:

```
axios.get('/api/transactions', {  
  headers: { Authorization: `Bearer ${token}` }  
});
```

---

## 5. Banco de Dados (MariaDB + Sequelize)

### 5.1. Estrutura e Mapeamento

- O Sequelize gera o schema através das **migrations**.
- **Seeders** preenchem categorias padrão.

Exemplo de Migration:

```
await queryInterface.createTable('users', {  
  id: { type: Sequelize.INTEGER, primaryKey: true, autoIncrement: true },  
  name: Sequelize.STRING,  
  email: Sequelize.STRING,  
  password_hash: Sequelize.STRING,  
  created_at: Sequelize.DATE,  
  updated_at: Sequelize.DATE  
});
```

---

## 6. Deploy e Ambiente de Execução

### 6.1. Requisitos

- Node.js  $\geq 18$
- Docker & Docker Compose
- MariaDB  $\geq 10.6$

### 6.2. Variáveis de Ambiente (.env)

DB\_HOST=localhost

DB\_USER=root

DB\_PASS=senha

DB\_NAME=meucontrolefinanceiro

JWT\_SECRET=chave\_super\_secreta

PORT=3333

### 6.3. Execução via Docker

docker-compose up --build

O serviço API estará disponível em:

 <http://localhost:3333>

e o frontend em:

 <http://localhost:5173>

---

## 7. Futuras Implementações (Versão 1.2)

| Recurso                  | Descrição                                                 |
|--------------------------|-----------------------------------------------------------|
| Notificações por e-mail  | Avisos de metas concluídas ou despesas acima do orçamento |
| Exportação de relatórios | Geração de PDFs via pdfkit                                |
| Dashboard de desempenho  | Gráficos comparativos entre meses                         |
| Testes automatizados     | Integração com Jest e Supertest                           |
| Modo escuro              | Tema personalizável                                       |

---

## 8. Conclusão

O **Meu Controle Financeiro** apresenta uma base sólida e modular para o controle financeiro pessoal, seguindo boas práticas de **segurança, escalabilidade e manutenibilidade**.

A arquitetura desacoplada facilita a integração com novas tecnologias e a expansão futura da aplicação.